

ตัวอย่างหน้าเพจหนึ่งเพจ แบบ Full Stack

หน้าเพจ : รายการหนังสือ (Book List Page)

ตัวอย่างโค้ดในแต่ละชั้น

My Library Admin

รายการหนังสือ + เพิ่มหนังสือ

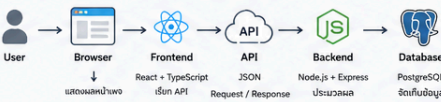
#	รูป	ชื่อหนังสือ	ผู้เขียน	ปีที่พิมพ์	จัดการ
1		Clean Code	Robert C. Martin	2008	<button>แก้ไข</button> <button>ลบ</button>
2		Design Patterns	Erich Gamma	1994	<button>แก้ไข</button> <button>ลบ</button>
3		JavaScript Guide	Kyle Simpson	2020	<button>แก้ไข</button> <button>ลบ</button>
4		The Pragmatic Programmer	Andrew Hunt	1999	<button>แก้ไข</button> <button>ลบ</button>

< 1 2 3 ... >

สิ่งที่เกิดขึ้นเมื่อเปิดหน้าเพจนี้

1. Browser ส่ง Request
GET https://myapp.com/api/books?page=1
2. Backend รับ Request
3. Query ข้อมูลจาก Database
4. ส่งข้อมูลกลับ (Response)
JSON
5. Frontend รับข้อมูลและแสดงผล

การไหลของข้อมูล (Data Flow)



สรุป

- ✓ ผู้ใช้เปิดหน้าเว็บ
- ✓ Frontend (React) รับข้อมูลแบบ JSON
- ✓ Backend ตรวจสอบสิทธิ์
- ✓ Database ค้นหาข้อมูลผ่าน ORM (Prisma)
- ✓ Backend ส่ง Response กลับเป็น JSON

การจำแนกชั้น (Layers)

1 PRESENTATION LAYER (UI)

ชั้นการแสดงผล

- React + Tailwind CSS
- แสดงตารางหนังสือ
- ปุ่มเพิ่ม/แก้ไข/ลบ
- Pagination

2 FRONTEND LOGIC LAYER

ชั้นตรรกะฝั่งหน้าเว็บ

- TypeScript, React Query (TanStack)
- เรียก API
- จัดการ State
- จัดการ Loading / Error

3 API / COMMUNICATION LAYER

ชั้นการสื่อสาร

- HTTP/HTTPS + JSON
- ส่ง Request
- รับ Response

4 BACKEND APPLICATION LAYER

ชั้นตรรกะฝั่งเซิร์ฟเวอร์

- Prisma ORM (Express.js)
- รับ Request
- ตรวจสอบสิทธิ์
- ประมวลผล Business Logic
- เรียกใช้ Service

5 DATA ACCESS LAYER (ORM)

ชั้นการเข้าถึงข้อมูล

- Query ข้อมูล
- Insert / Update / Delete
- Mapping Model

6 DATABASE LAYER

ชั้นฐานข้อมูล

- เก็บข้อมูลหนังสือ
- ความสัมพันธ์ของข้อมูล

1. Presentation Layer (React Component)

```
// BookList.jsx
export default function BookList() {
  const { data, isLoading } = useQuery({
    queryKey: ['books'],
    queryFn: () => fetch('/api/books?page=1').then(res => res.json())
  });
  return (
    <table>...แสดงตารางหนังสือ...</table>
  );
}
```

2. Frontend Logic (React Query)

```
const { data, isLoading, error } = useQuery({
  queryKey: ['books', page],
  queryFn: () => axios.get(`/api/books?page=${page}`)
    .then(res => res.data)
});
```

3. API Communication

```
// Request
GET /api/books?page=1
// Response
{
  "success": true,
  "data": [
    { "id": 1, "title": "Clean Code", "author": "Robert C. Martin", "year": 2008 },
    { "id": 2, "title": "Design Patterns", "author": "Erich Gamma", "year": 1994 }
  ],
  "total": 32
}
```

4. Backend (Express.js)

```
// books.routes.js
router.get('/books', async (req, res) => {
  const page = parseInt(req.query.page) || 1;
  const result = await bookService.getAllBooks(page);
  res.json({ success: true, data: result.data, total: result.total });
});
```

5. Data Access (Prisma)

```
// book.service.js
const getAllBooks = async (page) => {
  const take = 10;
  const skip = (page - 1) * take;
  const data = await prisma.book.findMany({
    skip, take, orderBy: { id: 'desc' }
  });
  const total = await prisma.book.count();
  return { data, total };
};
```

6. Database (PostgreSQL)

```
CREATE TABLE books (
  id SERIAL PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
  author VARCHAR(100) NOT NULL,
  year INT,
  created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);
```

id (PK)	SERIAL
title	VARCHAR
author	VARCHAR
year	INT
created_at	TIMESTAMP

เครื่องมือที่ใช้ (Tools)

Frontend

- React
- TypeScript
- Tailwind CSS
- React Query

Backend

- Node.js
- Express.js
- Prisma ORM

Database

- PostgreSQL

DevOps

- Docker
- Nginx
- GitHub Actions
- Vercel / AWS

อื่น ๆ

- VS Code
- Postman
- Git / GitHub